

</> DESVENDANDO O CÓDIGO

TAILWIND CSS

DO ZERO À INTERFACE RESPONSIVA

Guia atualizado para Tailwind CSS 4 + Vite

A proposta deste material é entender o Tailwind de forma prática: instalação, leitura das classes, responsividade e uso em projetos com React.

ATUALIZAÇÃO IMPORTANTE

Este material segue o fluxo atual do Tailwind CSS 4 com Vite. O comando antigo ``npx tailwindcss init -p`` não faz parte desta instalação.

01 • ENTENDENDO O TAILWIND CSS

O que é Tailwind CSS?

Tailwind CSS é um framework CSS baseado em classes utilitárias. Ele examina os arquivos HTML, os componentes JavaScript e outros templates do projeto, identifica as classes utilizadas e gera o CSS necessário para a interface.

Na prática, você descreve a aparência diretamente no elemento, sem precisar criar uma classe CSS personalizada para cada botão, card ou bloco da página.

CSS tradicional:

```
.card {  
  background: white;  
  padding: 20px;  
  border-radius: 12px;  
}
```

Com Tailwind:

```
<div class="bg-white p-5 rounded-xl">  
  ...  
</div>
```

A IDEIA QUE VALE GUARDAR

HTML/JSEX monta a estrutura. JavaScript ou React controla comportamento e dados. Tailwind cuida da aparência.

Para que ele serve?

- Criar interfaces sem manter um arquivo CSS enorme.
- Trabalhar com Flexbox e Grid por meio de classes utilitárias.
- Criar layouts responsivos com abordagem mobile first.
- Aplicar cores, tipografia, espaçamento, bordas, sombras e tamanhos.
- Trabalhar com estados como hover e focus.
- Manter a estilização próxima do elemento ou componente que está sendo construído.

02 • INSTALAÇÃO ATUAL COM VITE

React + Vite + Tailwind CSS 4

O fluxo atual recomendado para projetos Vite utiliza o pacote `tailwindcss` junto com o plugin `@tailwindcss/vite`. Não é necessário gerar `tailwind.config.js` nem `postcss.config.js` para começar.

Passo 1 — criar o projeto Vite

TERMINAL

```
npm create vite@latest meu-projeto
cd meu-projeto
npm install
```

NO ASSISTENTE DO VITE

Para a aula com React, escolha React e depois JavaScript. Se o projeto React já existe, pule diretamente para o Passo 2.

Passo 2 — instalar Tailwind CSS

TERMINAL

```
npm install tailwindcss @tailwindcss/vite
```

Passo 3 — configurar o plugin no Vite

Abra `vite.config.js` e deixe o plugin do React junto com o plugin do Tailwind:

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

Passo 4 — importar Tailwind no CSS

No arquivo CSS principal do projeto, normalmente `src/index.css`:

```
src/index.css
@import "tailwindcss";
```

O QUE MUDOU EM RELAÇÃO AOS TUTORIAIS ANTIGOS?

No Tailwind 4 com Vite, você não precisa usar `@tailwind base`, `@tailwind components` e `@tailwind utilities` para esta instalação. Também não precisa rodar `npx tailwindcss init -p`.

03 • PRIMEIRO TESTE

Confirmando se o Tailwind está funcionando

No `src/App.jsx`, use uma interface simples:

```
src/App.jsx
export default function App() {
  return (
    <main className="min-h-screen bg-slate-100 p-8">
      <h1 className="text-4xl font-bold text-yellow-500">
        React + Tailwind CSS
      </h1>

      <button className="
        mt-6 rounded-lg bg-black px-4 py-2
        font-semibold text-yellow-400
        hover:bg-slate-800
      ">
        Testar Tailwind
      </button>
    </main>
  )
}
```

Depois execute:

```
TERMINAL
npm run dev
```

CHECKPOINT

Se o fundo estiver claro, o título aparecer grande e amarelo e o botão ficar preto com texto amarelo, o Tailwind está carregado corretamente.

Se não funcionar

- Confirme se `@tailwindcss/vite` foi instalado.
- Confira se `tailwindcss()` está no array de plugins do `vite.config.js`.
- Confira se o CSS principal contém `@import "tailwindcss";`.
- Confirme se o CSS principal continua importado no `main.jsx`.

04 • COMO PENSAR NAS CLASSES

Você não precisa decorar Tailwind

A melhor forma de aprender é associar o que você já conhece de CSS ao nome da utility. Depois de algumas aulas, os nomes começam a ficar previsíveis.

TAILWIND	IDEIA EM CSS	O QUE FAZ
p-4	padding	espaço interno
mt-4	margin-top	margem superior
bg-black	background	fundo preto
text-yellow-400	color	texto amarelo
text-2xl	font-size	texto maior
font-bold	font-weight	negrito
rounded-xl	border-radius	cantos arredondados
shadow-md	box-shadow	sombra
flex	display: flex	Flexbox
grid	display: grid	CSS Grid
gap-6	gap	espaço entre itens
w-full	width: 100%	largura total

REGRA PRÁTICA

Pense primeiro: “qual CSS eu quero aplicar?”. Depois procure a utility correspondente. A documentação oficial funciona como seu dicionário.

05 • FLEXBOX, GRID E RESPONSIVIDADE

O coração de uma interface moderna

Flexbox

```
<header className="flex items-center justify-between p-4">
  <h1>Logo</h1>
  <nav>Menu</nav>
</header>
```

- `flex` ativa o Flexbox.
- `items-center` alinha os itens no eixo transversal.
- `justify-between` distribui espaço entre os elementos.
- `p-4` aplica espaçamento interno.

Grid responsivo

```
<section className="
  grid grid-cols-1 gap-6
  sm:grid-cols-2
  lg:grid-cols-3
">
  ...
</section>
```

- `grid-cols-1` → 1 coluna por padrão.
- `sm:grid-cols-2` → 2 colunas em telas a partir do breakpoint `sm`.
- `lg:grid-cols-3` → 3 colunas em telas maiores.
- `gap-6` → espaço entre os cards.

MOBILE FIRST

A classe sem prefixo vale primeiro para telas menores. Os prefixos `sm:`, `md:` e `lg:` entram conforme a tela cresce.

06 • TAILWIND DENTRO DO REACT

Do HTML para o componente

Em HTML puro usamos `class`. No JSX do React usamos `className`.

ProductCard.jsx

```
function ProductCard({ produto }) {  
  return (  
    <article className="  
      rounded-xl bg-white p-5  
      shadow-md transition  
      hover:-translate-y-1 hover:shadow-xl  
    ">  
      <h2 className="text-xl font-bold">  
        {produto.nome}  
      </h2>  
  
      <p className="mt-2 text-slate-500">  
        {produto.descricao}  
      </p>  
  
      <button className="  
        mt-4 rounded-lg bg-black  
        px-4 py-2 text-yellow-400  
      ">  
        Ver produto  
      </button>  
    </article>  
  )  
}
```

SEPARE AS RESPONSABILIDADES

React fornece o componente e recebe os dados pela prop `produto`. Tailwind define a apresentação visual do card.

07 • ESTADOS, EFEITOS E VARIANTES

Quando as classes mudam de acordo com a situação

Tailwind permite aplicar utilities apenas em determinados estados.

```
<button className="
  bg-yellow-400 text-black
  px-4 py-2 rounded-lg
  hover:bg-yellow-300
  focus:outline-none
  transition
">
  Comprar
</button>
```

- `hover:` aplica o estilo quando o mouse passa sobre o elemento.
- `focus:` aplica o estilo quando o elemento recebe foco.
- `transition` suaviza alterações visuais.
- Prefixos responsivos e estados podem ser combinados com utilities.

08 • ERROS COMUNS E ATALHOS

Evite misturar versões

Erro clássico: `npx tailwindcss init -p`

Esse comando aparece em muitos vídeos e materiais antigos do Tailwind 3. No fluxo atual com Tailwind CSS 4 + Vite, ele não faz parte da instalação.

CORRETO NO FLUXO DESTE GUIA

```
npm install tailwindcss @tailwindcss/vite
```

Outros erros comuns

- Instalar Tailwind 4 e seguir um tutorial escrito para Tailwind 3.
- Esquecer de adicionar `tailwindcss()` no `vite.config.js`.
- Esquecer `@import "tailwindcss";` no CSS principal.
- Usar `class` dentro do JSX em vez de `className`.
- Copiar dezenas de utilities sem testar o efeito de cada uma.

09 • COLA RÁPIDA

As utilities que mais aparecem no começo

LAYOUT	<code>flex grid hidden block</code>
ALINHAMENTO	<code>items-center justify-center justify-between</code>
GRID	<code>grid-cols-1 grid-cols-2 grid-cols-3 gap-4 gap-6</code>
ESPAÇAMENTO	<code>p-4 p-6 px-4 py-2 mt-4 mb-4</code>
TEXTO	<code>text-sm text-xl text-2xl font-semibold font-bold</code>
CORES	<code>bg-black bg-white bg-slate-100 text-yellow-400</code>
TAMANHO	<code>w-full max-w-6xl min-h-screen</code>
EFEITOS	<code>rounded-lg rounded-xl shadow-md transition</code>
ESTADOS	<code>hover:bg-* hover:shadow-* focus:*</code>
RESPONSIVO	<code>sm:* md:* lg:*</code>

NÃO DECORE TUDO

Domine as classes que aparecem no projeto da aula. Para o restante, consulte a documentação. Framework muda; saber ler documentação continua sendo a habilidade principal.

10 • ROTEIRO PARA SUA AULA / LIVE

Uma sequência simples que funciona

- | | |
|----|--|
| 01 | Mostrar o projeto pronto — Abra com o resultado final para dar contexto visual. |
| 02 | Explicar o que é Tailwind — Utility-first, classes e diferença para CSS tradicional. |
| 03 | Instalar com Vite — <code>`tailwindcss`</code> + <code>`@tailwindcss/vite`</code> . |
| 04 | Configurar o plugin — Adicionar <code>`tailwindcss()`</code> ao <code>`vite.config.js`</code> . |
| 05 | Importar o CSS — Usar <code>`@import "tailwindcss";`</code> . |
| 06 | Criar primeiro bloco — Cor, padding, texto e borda. |
| 07 | Entrar em Flex e Grid — Construir o layout progressivamente. |
| 08 | Adicionar responsividade — Comparar celular, tablet e desktop. |
| 09 | Adicionar hover — Mostrar interação visual. |
| 10 | Levar para React — Mostrar <code>`className`</code> e componentes. |

RESUMO FINAL

Tailwind CSS 4 com Vite hoje pode começar com três ideias: instalar os dois pacotes, registrar o plugin no Vite e importar ``tailwindcss`` no CSS. Depois disso, o aprendizado é dominar as utilities necessárias para construir a interface.